Chapter 6 | Exercise solutions with Python

Training and Generating with a Small Transformer

This notebook solves Exercises 1, 3, and 6–12, including all four optional experiments. It checks shifted targets, loss aggregation, stopping, training, length extrapolation, model selection, leakage, and the decoder cache. Exercises 2, 4, and 5 are in the written solutions.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements-ch06.txt`. Run the cells in order. The exercises train small models on the CPU and include their saved outputs.

```
from pathlib import Path
import sys

import torch
from torch import nn
from torch.nn import functional as F

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch06_training" / "training.py"
    )
    if model_path.is_file():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError("Open this notebook inside transformers-book.")

torch.set_num_threads(1)
```

1. Align a padded batch

Start by reversing each source's colors. For `red green blue` the target colors are `blue green red`. For `red blue` they are `blue red`. Place `BOS` before the colors in the decoder input and `EOS` after them in the labels. At position j, the decoder input is the token just before the label in the completed answer. Right-pad each array to its own widest row. Here the source width is 3 and both target arrays have width 4.

Example	Source IDs as tokens	Decoder input	Label row
1	`red green blue`	`BOS blue green red`	`blue green red EOS`
2	`red blue PAD`	`BOS blue red PAD`	`blue red EOS PAD`

A 1 marks a valid input or an evaluated label. Build the source and target input masks from their inputs. Build the loss-selection mask from the **labels**. The two target masks happen to agree because the shift replaces one boundary token with another, leaving the same number of valid slots in each row.

Example	Source input validity	Decoder input validity	Loss selection
1	`1 1 1`	`1 1 1 1`	`1 1 1 1`
2	`1 1 0`	`1 1 1 0`	`1 1 1 0`

Read across each row of the causal grid to find the keys allowed for one query. The triangle allows key k for query q when $k \leq q$. At position 2 of example 1, the current input is `blue` and the label is `green`. The diagonal lets the query read `blue`, which is already available. The decoder input `green` is one position later, outside that row's allowed keys.

Query position	Key 1	Key 2	Key 3	Key 4
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0
4	1	1	1	1

All target keys in example 1 are valid. In example 2 the key-validity mask removes column 4 from the triangle. Its padded query in row 4 can still read the first three keys. The loss-selection mask skips the output at that position because its label is `PAD`.

```
import math

from examples.ch05_transformer.model import Transformer, build_masks
from examples.ch06_training.data import BOS, EOS, PAD, VOCAB, collate
from examples.ch06_training.run_experiment import loss_sanity_example

RED, GREEN, BLUE = (VOCAB.index(c) for c in ("red", "green", "blue"))
batch_1 = collate([(RED, GREEN, BLUE), (RED, BLUE)])
loss_selected_1 = batch_1.target != PAD

assert batch_1.source.tolist() == [
    [RED, GREEN, BLUE], [RED, BLUE, PAD],
]
assert batch_1.decoder_input.tolist() == [
    [BOS, BLUE, GREEN, RED], [BOS, BLUE, RED, PAD],
]
assert batch_1.target.tolist() == [
    [BLUE, GREEN, RED, EOS], [BLUE, RED, EOS, PAD],
]
assert batch_1.source_valid.int().tolist() == [[1, 1, 1], [1, 1, 0]]
assert batch_1.target_valid.int().tolist() == [[1, 1, 1, 1], [1, 1, 1, 0]]
assert loss_selected_1.int().tolist() == [[1, 1, 1, 1], [1, 1, 1, 0]]

print("Source IDs      ", batch_1.source.tolist())
print("Decoder input   ", batch_1.decoder_input.tolist())
print("Labels          ", batch_1.target.tolist())
print("Source valid    ", batch_1.source_valid.int().tolist())
print("Decoder valid   ", batch_1.target_valid.int().tolist())
print("Loss selection  ", loss_selected_1.int().tolist())
```

OUTPUT

```
Source IDs       [[3, 4, 5], [3, 5, 0]]
Decoder input    [[1, 5, 4, 3], [1, 5, 3, 0]]
Labels           [[5, 4, 3, 2], [5, 3, 2, 0]]
Source valid     [[1, 1, 1], [1, 1, 0]]
Decoder valid    [[1, 1, 1, 1], [1, 1, 1, 0]]
Loss selection   [[1, 1, 1, 1], [1, 1, 1, 0]]
```

```
_, decoder_allowed_1 = build_masks(
    batch_1.source_valid, batch_1.target_valid
)
assert decoder_allowed_1.shape == (2, 1, 4, 4)
assert decoder_allowed_1[0, 0].int().tolist() == [
    [1, 0, 0, 0], [1, 1, 0, 0], [1, 1, 1, 0], [1, 1, 1, 1],
]
assert decoder_allowed_1[1, 0].int().tolist() == [
    [1, 0, 0, 0], [1, 1, 0, 0], [1, 1, 1, 0], [1, 1, 1, 0],
]
print("Allowed keys per query, example 2")
for row in decoder_allowed_1[1, 0].int().tolist():
    print(row)

sanity_1 = loss_sanity_example()
assert loss_selected_1.sum().item() == sanity_1["tokens"] == 7
assert math.isclose(sanity_1["loss_sum"], 7 * math.log(9))
assert math.isclose(sanity_1["perplexity"], 9)
print(f"Evaluated labels  {sanity_1['tokens']}")
print(f"Loss sum          {sanity_1['loss_sum']:.6f}")
print(f"Mean NLL          {sanity_1['nll']:.6f}")
print(f"Perplexity        {sanity_1['perplexity']:.6f}")
```

OUTPUT

```
Allowed keys per query, example 2
[1, 0, 0, 0]
[1, 1, 0, 0]
[1, 1, 1, 0]
[1, 1, 1, 0]
Evaluated labels  7
Loss sum          15.380572
Mean NLL          2.197225
Perplexity        9.000000
```

The count is $(3 + 1) + (2 + 1) = 7$ because each row contributes its colors and one `EOS` label. `BOS` is supplied as an input. The sanity check gives all nine vocabulary classes equal logits, so the probability of each correct token is $1/9$. Seven valid labels give a total loss $7 \ln 9$, a mean $\ln 9$, and perplexity $\exp(\ln 9) = 9$.

3. Aggregate before dividing

Count the four evaluated tokens as four equal contributions. Three of them are in the first batch, and together they contribute 1.2 units of negative log-probability. The second batch contains one token with loss 1.0. The corpus mean is

$$NLL = \frac{1.2 + 1.0}{3 + 1} = \frac{2.2}{4} = 0.55, \qquad perplexity = e^{0.55} \approx 1.733253.$$

The two batch means are $1.2/3 = 0.4$ and $1.0/1 = 1.0$. Taking their unweighted average gives $(0.4 + 1.0)/2 = 0.7$, whose exponential is about 2.013753. This gives the single token in the second batch half the total influence, even though it accounts for one quarter of the labels. Weighting the batch means by the valid token counts recovers $(3 \cdot 0.4 + 1 \cdot 1.0)/4 = 0.55$.

We can check partitioning with a fixed float64 model in evaluation mode. The three sources below have lengths 3, 6, and 4. Their evaluated label counts are 4, 7, and 5, including `EOS`. A batch size of 1 separates all three examples. A batch size of 2 places the first two together and leaves the last one alone. Both calls use a generation cap of 8. The cap applies to the generated answers, whereas the NLL comes from the labeled predictions using their reference prefixes.

```
from examples.ch06_training.training import evaluate, token_loss

def make_ch06_checks_model():
    with torch.random.fork_rng(devices=[]):
        torch.manual_seed(23)
        model = Transformer(
            len(VOCAB), len(VOCAB), d_model=8, n_heads=2, d_ff=16,
            n_encoder_layers=2, n_decoder_layers=2, dropout=0.0,
            max_length=32,
        )
    return model.double().eval()

model_3 = make_ch06_checks_model()
sequences_3 = [(3, 4, 5), (4, 4, 6, 6, 5, 3), (5, 3, 6, 4)]
cap_3 = 8
evaluation_3 = [
    evaluate(
        model_3, sequences_3, batch_size=size,
        generation_max_new_tokens=cap_3,
    )
    for size in (1, 2)
]
print("batch size   labels   corpus NLL       perplexity")
for size, result in zip((1, 2), evaluation_3):
    print(f"{size:10d}   {result['tokens']:6d}   "
          f"{result['nll']:.12f}   {result['perplexity']:.8f}")

assert evaluation_3[0]["tokens"] == evaluation_3[1]["tokens"] == 16
difference_3 = abs(evaluation_3[0]["nll"] - evaluation_3[1]["nll"])
assert difference_3 <= 1e-12
print(f"Absolute NLL difference  {difference_3:.3e}")
```

OUTPUT

```
batch size   labels   corpus NLL       perplexity
         1       16   2.419337529505   11.23841173
         2       16   2.419337529505   11.23841173
Absolute NLL difference  0.000e+00
```

```
sums_3, counts_3 = [], []
with torch.no_grad():
    for subset in (sequences_3[:2], sequences_3[2:]):
        batch_3 = collate(subset)
        logits_3 = model_3(batch_3.source, batch_3.decoder_input)
        total_3, count_3 = token_loss(logits_3, batch_3.target)
        sums_3.append(total_3.item())
        counts_3.append(count_3.item())

mean_3 = sum(sums_3) / sum(counts_3)
unweighted_3 = sum(s / n for s, n in zip(sums_3, counts_3)) / 2
assert counts_3 == [11, 5]
assert math.isclose(mean_3, evaluation_3[1]["nll"], abs_tol=1e-12)
print("Batch loss sums  ", [round(s, 8) for s in sums_3])
print("Valid counts     ", counts_3)
print(f"Sum / total count         {mean_3:.12f}")
print(f"Unweighted batch average  {unweighted_3:.12f}")
```

```
OUTPUT
Batch loss sums   [24.04506736, 14.66433311]
Valid counts      [11, 5]
Sum / total count         2.419337529505
Unweighted batch average  2.559390918409
```

Batching changes where padding is stored and how the floating-point operations are grouped. Attention masks exclude padded keys, and loss selection skips padded labels. The same real labels contribute to the corpus mean in each partition. A tolerance of 10^{-12} accommodates the small arithmetic differences in this float64 check.

6. Test the stopping rule

Use the stop tests in `tests/test_training.py` as a model. Script the argmax choice at each step by giving that vocabulary class a large score and giving the other classes small scores. We can then check the stopping behavior without relying on a trained model to produce the particular token we need.

Predict the returned arrays before running the checks. `EOS` has ID 2, `PAD` has ID 0, and `green` has ID 4. The initial `BOS` is an input and is absent from the returned array.

Script	Expected tokens	Expected IDs	Decode calls
One row, immediate ending	`[[EOS]]`	`[[2]]`	1
One row, green through the cap	`[[green, green, green]]`	`[[4,4,4]]`	3
Two rows, one ends first	`[[EOS,PAD,PAD], [green,green,EOS]]`	`[[2,0,0], [4,4,2]]`	3

Call `greedy_decode` with `cached=False` so it calls the patched `decoding.decode_prefix`. The stub receives the accumulated prefix. Subtract one from its width to get the zero-based generation step because its first slot contains `BOS`. Fill only the last logits row, which is the row the generation loop uses for its next choice.

```
from unittest.mock import patch
from examples.ch06_training import decoding

def scripted_prefix(schedule):
    def stub(model, prefix, memory, source_allowed):
        step = prefix.shape[1] - 1
        chosen = prefix.new_tensor(schedule[step]).unsqueeze(-1)
        logits = memory.new_full((*prefix.shape, len(VOCAB)), -100.0)
        logits[:, -1].scatter_(1, chosen, 10.0)
        return logits
    return stub

scripts_6 = (
    ("Immediate EOS", [[EOS]], [[EOS]]),
    ("Green to cap", [[GREEN], [GREEN], [GREEN]],
     [[GREEN, GREEN, GREEN]]),
    ("Mixed", [[EOS, GREEN], [EOS, GREEN], [EOS, EOS]],
     [[EOS, PAD, PAD], [GREEN, GREEN, EOS]]),
)
```

```
model_6 = make_ch06_checks_model()
for name, script, expected in scripts_6:
    source_6 = batch_1.source[:len(expected)]
    with patch.object(
        decoding, "decode_prefix", side_effect=scripted_prefix(script)
    ) as traced:
        generated_6 = decoding.greedy_decode(
            model_6, source_6, max_new_tokens=3, cached=False,
        )
    assert generated_6.tolist() == expected
    assert traced.call_count == len(script)
    print(f"{name:15s}  {generated_6.tolist()}  calls={traced.call_count}")

    if name == "Mixed":
        finished_inputs_6 = [
            call.args[1][0, -1].item() for call in traced.call_args_list
        ]
        assert finished_inputs_6 == [BOS, EOS, EOS]
        print("Row 1 inputs during the three calls:", finished_inputs_6)
```

OUTPUT

```
Immediate EOS    [[2]]  calls=1
Green to cap     [[4, 4, 4]]  calls=3
Mixed            [[2, 0, 0], [4, 4, 2]]  calls=3
Row 1 inputs during the three calls: [1, 2, 2]
```

The generation loop stops when all rows have emitted `EOS` or when the cap has been reached. The second case reaches the cap with three colors and no ending, so its termination flag is false. In the mixed case, the first row emits `EOS` on the first call. The loop keeps calling the decoder while the second row is active. The completed row receives `EOS` as input during those calls, while the output stores `PAD` after the first `EOS` to keep the array rectangular.

7. Overfit before scaling

Choose four source sequences and keep presenting the same batch. For example, the answer to *red green blue* is *blue green red*, followed by EOS, the end-of-sequence token. The exercise isolates the link between decoder inputs and prediction targets. A correctly wired model should be able to memorize a few examples, and we can check that memorization by asking it to generate the answers.

At each decoder position, **teacher forcing** supplies the previous reference token as input. BOS, the beginning-of-sequence token, occupies the first position. The model must predict the next token in the row.

Decoder position	1	2	3	4
Shifted input	BOS	blue	green	red
Label to predict	blue	green	red	EOS
Deliberately unshifted input	blue	green	red	EOS

In the unshifted experiment, the current input reveals its own label. The causal mask permits the diagonal, so the current position can use that input. The decoder's residual paths also carry information from the current input embedding. Copying it can earn a low training loss with little need to learn the reversal.

The two runs begin from fresh models with the same seed and default initialization. Each uses width 16, two heads, FFN width 32, one layer per stack, zero dropout, and Adam with rate `0.01`. Each update uses all four examples. The generation cap is six, enough to emit four colors and EOS with one position to spare. The scores and generation checks here use the four training examples.

```
from examples.ch05_transformer.model import Transformer
from examples.ch06_training.data import Batch, VOCAB, collate, words
from examples.ch06_training.decoding import greedy_decode
from examples.ch06_training.training import (
    exact_matches, token_loss, train_step,
)

sources_overfit = [
    (3, 4, 5), (6, 5, 3), (7, 4, 8, 6), (8, 6, 5, 4),
]
shifted = collate(sources_overfit)
unshifted = Batch(
    shifted.source, shifted.target.clone(), shifted.target,
)
for source, target in zip(shifted.source, shifted.target):
    print(f"{words(source):27} -> {words(target, True)}")
```

OUTPUT

```
red green blue              -> blue green red <eos>
yellow blue red             -> red blue yellow <eos>
white green black yellow    -> yellow black green white <eos>
black yellow blue green     -> green blue yellow black <eos>
```

`collate` produces the shifted inputs and labels. The experimental batch copies the labels into the decoder input and keeps the same sources and labels. Its right-padding pattern also stays the same. Both models use the canonical `train_step`, including its gradient norm cap. At each checkpoint, switch to evaluation mode and recompute mean token loss with the input arrangement used in that run. Then start greedy decoding from BOS without supplying the reference sequence.

```
def overfit_checkpoint(model, batch):
    model.eval()
    with torch.no_grad():
        logits = model(batch.source, batch.decoder_input)
        total, count = token_loss(logits, batch.target)
    generated = greedy_decode(
        model, batch.source, max_new_tokens=6, cached=False,
    )
    return {
        "nll": (total / count).item(),
        "matches": exact_matches(generated, batch.target),
        "rows": [words(row, include_special=True) for row in generated],
    }


def fit_four_examples(batch):
    with torch.random.fork_rng(devices=[]):
        torch.manual_seed(2026)
        model = Transformer(
            len(VOCAB), len(VOCAB), d_model=16, n_heads=2, d_ff=32,
            n_encoder_layers=1, n_decoder_layers=1, dropout=0.0,
        )
    optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
    snapshots = {0: overfit_checkpoint(model, batch)}
    for step in range(1, 201):
        train_step(model, optimizer, batch)
        if step in (20, 75, 200):
            snapshots[step] = overfit_checkpoint(model, batch)
    return snapshots
```

```
torch.set_num_threads(1)
shifted_trace = fit_four_examples(shifted)
unshifted_trace = fit_four_examples(unshifted)

assert shifted_trace[0]["rows"] == unshifted_trace[0]["rows"]
print("Update  Shifted NLL  Exact     Unshifted NLL  Exact")
for step, correct in shifted_trace.items():
    leaky = unshifted_trace[step]
    print(
        f"{step:>6}  {correct['nll']:>11.6f}  {correct['matches']}/4"
        f"       {leaky['nll']:>13.6f}  {leaky['matches']}/4"
    )

for label, trace in (
    ("Shifted", shifted_trace), ("Unshifted", unshifted_trace),
):
    for step in (20, 200):
        print(f"\n{label}, generated answers at update {step}")
        for row in trace[step]["rows"]:
            print(f"  {row}")
```

```
OUTPUT
Update  Shifted NLL  Exact     Unshifted NLL  Exact
     0     2.365434  0/4            2.315844  0/4
    20     0.219827  4/4            0.083590  0/4
    75     0.003179  4/4            0.003109  0/4
   200     0.000894  4/4            0.000953  0/4

Shifted, generated answers at update 20
  blue green red <eos>
  red blue yellow <eos>
  yellow black green white <eos>
  green blue yellow black <eos>

Shifted, generated answers at update 200
  blue green red <eos>
  red blue yellow <eos>
  yellow black green white <eos>
  green blue yellow black <eos>

Unshifted, generated answers at update 20
  green green green green green green
  green green green green green green
  green green green green green green
  green green green green green green

Unshifted, generated answers at update 200
  green green green green green green
  green green green green green green
  green green green green green green
  green green green green green green
```

The two models generate the same rows at update zero because both start from the same weights and both generation calls supply the same sources and BOS. Their initial losses differ because their scoring inputs differ. By update 20, the shifted model generates all four reversals and their EOS tokens. The unshifted run's training NLL is already lower at that update, yet its generated rows repeat *green* until the cap. At update 200, both training losses are about `0.001`, and the unshifted model still generates zero exact matches. It learned to make predictions with the answer in the input. During generation it has only the source, BOS, and its own outputs. The later exercises construct fresh shifted batches with `collate`.

8. Probe the limits of the task

A model may learn to reverse unfamiliar arrangements of familiar lengths, yet struggle when asked to reverse longer sequences. Keep the model fixed and measure those two abilities separately. First use the chapter's test split, whose lengths 3 through 6 occur in training. Then use a new test split of lengths 7 and 8. The colors, target format, and evaluation rules are shared.

To make the checkpoint available on a fresh clone, rerun the chapter's reference recipe in a temporary directory. It uses seed 2026 for the model, batch sampling, and data generation. The model has width 64, four heads, feed-forward width 128, two layers in each stack, and zero dropout. The recipe initializes the matrices with Xavier uniform, uses Adam with learning rate 0.001 and clips the gradient norm at 1. It performs 2,000 updates with batches of 64 and checks validation NLL every 100 updates, including the initial checkpoint at update zero. The lowest validation token NLL selects the saved weights.

The cell below reports the elapsed time on one CPU thread. Training is seeded within the recorded environment. Results can vary across PyTorch versions, including the selected update. After reloading the checkpoint, every comparison below uses those same weights in evaluation mode.

```
from contextlib import redirect_stdout
from io import StringIO
from tempfile import TemporaryDirectory

from examples.ch06_training.run_experiment import (
    run as run_reference_8,
    split_fingerprint as fingerprint_8,
)
from examples.ch06_training.training import (
    evaluate as evaluate_8,
    load_checkpoint as load_checkpoint_8,
)

with TemporaryDirectory(prefix="ch06-ex8-") as directory_8:
    with redirect_stdout(StringIO()):
        reference_8 = run_reference_8(
            directory_8, steps=2000, seed=2026, data_seed=2026,
            batch_size=64, validation_every=100,
        )
    model_8, checkpoint_8 = load_checkpoint_8(
        Path(directory_8) / "best.pt"
    )

print("Selected update", checkpoint_8["step"])
print("Validation NLL", f'{checkpoint_8["validation"]["nll"]:.8f}')
print("PyTorch", torch.__version__)
print("Parameter dtype", next(model_8.parameters()).dtype)
print("CPU threads", torch.get_num_threads())
print("Training and evaluation seconds",
      f'{reference_8["elapsed_seconds"]:.1f}')
```

```
OUTPUT
Selected update 1700
Validation NLL 0.00419161
PyTorch 2.9.0
Parameter dtype torch.float32
CPU threads 1
Training and evaluation seconds 29.0
```

Reconstruct the original splits from the checkpoint's data seed and verify their stored fingerprints. Create the longer split with the exercise's requested sizes and use its 64 test sequences. Its training and validation portions are simply part of the split construction. We use neither for fitting or selection. There are 16 original test sequences per length and 32 longer test sequences per length.

Each evaluation call uses the same cap of 10 generated tokens. A source with eight colors needs eight output colors plus EOS, so that cap gives it room to finish. Using the same cap keeps the stopping allowance from changing across groups.

Token NLL averages $-\ln p$ over valid labels, including EOS, under the reference prefix. Token accuracy also uses those prefixes. In free-running greedy decoding, each selected token instead supplies the next decoder input. Exact match requires every target color in order and the final EOS. The EOS count only measures whether decoding ended by the cap.

```
from examples.ch06_training.data import make_splits as make_splits_8

record_8 = checkpoint_8["experiment"]
seed_8 = record_8["data_seed"]
original_8 = make_splits_8(seed=seed_8)
for name_8, split_8 in zip(
    ("train", "validation", "test"), original_8
):
    assert fingerprint_8(split_8) == record_8["split_sha256"][name_8]
test_8 = original_8[2]
_, _, longer_test_8 = make_splits_8(
    seed=seed_8, min_length=7, max_length=8,
    n_train=8, n_validation=8, n_test=64,
)
cap_8 = 10

def report_lengths_8(model, sequences):
    def report(label, group):
        metrics = evaluate_8(
            model, group, generation_max_new_tokens=cap_8
        )
        size = metrics["sequences"]
        hits = round(metrics["exact_match"] * size)
        eos = round(metrics["termination_rate"] * size)
        print(
            f'{label:>6} {size:5d} {metrics["tokens"]:6d} '
            f'{metrics["nll"]:9.6f} '
            f'{metrics["token_accuracy"]:9.2%} '
            f'{hits:4d}/{size:<3d} {eos:3d}/{size:<3d}'
        )

    print("Length  Seq. Tokens       NLL  Tok. acc.     Exact     EOS")
    for length in sorted(set(map(len, sequences))):
        group = [source for source in sequences if len(source) == length]
        report(length, group)
    report("all", sequences)

print("Held-out lengths seen in training")
report_lengths_8(model_8, test_8)
print("\nHeld-out lengths beyond training")
report_lengths_8(model_8, longer_test_8)
```

OUTPUT

```
Held-out lengths seen in training
Length  Seq. Tokens       NLL  Tok. acc.     Exact     EOS
     3    16     64  0.000190   100.00%   16/16   16/16
     4    16     80  0.000117   100.00%   16/16   16/16
     5    16     96  0.009889    98.96%   15/16   16/16
     6    16    112  0.039511    98.21%   14/16   16/16
   all    64    352  0.015330    99.15%   61/64   64/64

Held-out lengths beyond training
Length  Seq. Tokens       NLL  Tok. acc.     Exact     EOS
     7    32    256  1.466366    73.83%    2/32   32/32
     8    32    288  4.788201    32.64%    0/32   32/32
   all    64    544  3.224984    52.02%    2/64   64/64
```

To see which errors contribute to the exact-match gap, decode each test batch with the same cap. Compare the emitted colors before the first EOS with the reversed source. In the examples below, a substitution is a wrong color at a position when the predicted and expected color sequences have equal length. That rule avoids calling every shifted position a substitution when an output ends early. An excess repetition means a color appears at least twice in the generated sequence and more often than in the reference. A legitimate repeat can be part of the correct answer.

Check whether EOS arrives before or after the required number of colors, or never arrives within the cap. The counters can overlap. For each kind of error that occurs, show its first example in test order and keep EOS visible in the output.

```
from collections import Counter

from examples.ch06_training.data import (
    EOS as EOS_8,
    collate as collate_8,
    words as words_8,
)
from examples.ch06_training.decoding import (
    greedy_decode as greedy_decode_8,
)

def trace_errors_8(model, sequences):
    generated = greedy_decode_8(
        model, collate_8(sequences).source, max_new_tokens=cap_8
    )
    counts = Counter()
    samples = {}
    for source, row in zip(sequences, generated.tolist()):
        expected = list(reversed(source))
        ended = EOS_8 in row
        colors = row[:row.index(EOS_8)] if ended else row
        difference = len(colors) - len(expected)
        excess_repeat = any(
            copies > max(1, expected.count(color))
            for color, copies in Counter(colors).items()
        )
        errors = {
            "Substitution at matching length": (
                difference == 0 and colors != expected
            ),
            "Excess repetition": excess_repeat,
            "Early EOS": ended and difference < 0,
            "Late EOS": ended and difference > 0,
            "No EOS by cap": not ended,
        }
        for kind, occurred in errors.items():
            counts[kind] += int(occurred)
            if occurred:
                samples.setdefault(kind, (source, expected, row))
    return counts, samples
```

```
for name_8, sequences_8 in (
    ("In range", test_8), ("Extrapolation", longer_test_8)
):
    print(name_8)
    counts_8, samples_8 = trace_errors_8(model_8, sequences_8)
    for kind_8, count_8 in counts_8.items():
        print(f"{kind_8}: {count_8}/{len(sequences_8)}")
        if count_8:
            source_8, expected_8, row_8 = samples_8[kind_8]
            print("  Source   ", words_8(source_8))
            print("  Reference", words_8(expected_8 + [EOS_8], True))
            print("  Generated", words_8(row_8, True))
    print()
```

OUTPUT

```
In range
Substitution at matching length: 3/64
  Source    black black red black black
  Reference black black red black black <eos>
  Generated black black black red black <eos>
Excess repetition: 1/64
  Source    green white white yellow yellow green
  Reference green yellow yellow white white green <eos>
  Generated green yellow white white white green <eos>
Early EOS: 0/64
Late EOS: 0/64
No EOS by cap: 0/64

Extrapolation
Substitution at matching length: 2/64
  Source    black white yellow white yellow yellow black
  Reference black yellow yellow white yellow white black <eos>
  Generated yellow black yellow white yellow white black <eos>
Excess repetition: 7/64
  Source    white yellow black black red red green red
  Reference red green red red black black yellow white <eos>
  Generated red red black black black yellow <eos>
Early EOS: 60/64
  Source    black red green black blue yellow blue
  Reference blue yellow blue black green red black <eos>
  Generated blue yellow black black green red <eos>
Late EOS: 0/64
No EOS by cap: 0/64
```

In the saved run, 61 of 64 in-range reversals are exact. The model emits the correct number of colors on all 64, but a misplaced color or an excess repeat can still break the reversal. In the displayed repetition example, the model emits three copies of white where the reference has two, and one copy of yellow where the reference has two.

On longer sources, exact match drops to 2 of 64. All rows emit EOS, yet 60 stop early. The displayed early-ending example has six colors before EOS, with seven required. The same-length example instead exchanges the first two colors and stops on schedule. The longer repetition example overuses black and stops early, so it contributes to both counts.

The high EOS rate therefore records successful stopping in the narrow sense of ending by the cap. Token accuracy gives partial credit under the correct prefix. Exact match checks that the model keeps every decision on track under its own generated prefix. These results describe this split and the selected checkpoint.

9. Separate validation from testing (optional experiment)

Suppose two learning rates produce different trajectories. We need a way to select both a rate and an update before looking at the test set. Use the chapter's training, validation, and test splits with data seed 2026. Train on the training split only and rank checkpoints by validation token NLL. The test split supplies one final assessment of the selected checkpoint.

To keep this comparison quick, train a smaller model with width 32, four heads, feed-forward width 64, one encoder layer, one decoder layer, and dropout zero. Both runs start from copies of the same Xavier-initialized model. Both use Adam with default betas, batch size 32, the same seeded sequence of training batches, and gradient norm cap 1 through `train_step`. Only the learning rate differs.

Compare 0.001 with 0.004 over 240 updates each. Record validation at updates 0, 60, 120, 180, and 240 with cap 10. Include update zero as a valid checkpoint. Select the smallest validation NLL across both runs. If scores tie, prefer the earlier update, then the smaller learning rate. The rule and evaluation schedule are fixed before training.

```
from copy import deepcopy
import random

from examples.ch05_transformer.model import (
    Transformer as Transformer_9,
)
from examples.ch06_training.data import (
    VOCAB as VOCAB_9,
    collate as collate_9,
    make_splits as make_splits_9,
)
from examples.ch06_training.training import (
    evaluate as evaluate_9,
    train_step as train_step_9,
)

train_9, validation_9, test_9 = make_splits_9(seed=2026)
torch.set_num_threads(1)
with torch.random.fork_rng(devices=[]):
    torch.manual_seed(2026)
    initial_9 = Transformer_9(
        len(VOCAB_9), len(VOCAB_9), d_model=32, n_heads=4,
        d_ff=64, n_encoder_layers=1, n_decoder_layers=1,
        dropout=0.0, max_length=64,
    )
    for parameter in initial_9.parameters():
        if parameter.ndim > 1:
            nn.init.xavier_uniform_(parameter)

cap_9 = 10
rates_9 = (0.001, 0.004)
baseline_9 = evaluate_9(
    initial_9, validation_9, generation_max_new_tokens=cap_9
)
```

Each rate receives a fresh optimizer, so Adam's state starts empty both times. Reset the training sampler to the same seed before each run. A validation result has no effect on the next training batch. Copy the winning model when it improves on the best candidate. Otherwise, further updates would overwrite the parameters of the checkpoint we meant to retain. The loop never calls the test set.

```
best_key_9 = (baseline_9["nll"], 0, min(rates_9))
winner_9 = deepcopy(initial_9)
history_9 = [(rate, 0, baseline_9) for rate in rates_9]

for rate_9 in rates_9:
    candidate_9 = deepcopy(initial_9)
    optimizer_9 = torch.optim.Adam(candidate_9.parameters(), lr=rate_9)
    sampler_9 = random.Random(2026)
    for update_9 in range(1, 241):
        batch_9 = collate_9(sampler_9.sample(train_9, 32))
        train_step_9(candidate_9, optimizer_9, batch_9)
        if update_9 % 60 != 0:
            continue
        scores_9 = evaluate_9(
            candidate_9, validation_9,
            generation_max_new_tokens=cap_9,
        )
        history_9.append((rate_9, update_9, scores_9))
        key_9 = (scores_9["nll"], update_9, rate_9)
        if key_9 < best_key_9:
            best_key_9 = key_9
            winner_9 = deepcopy(candidate_9)
```

The history displays the loss used for selection along with exact match and EOS rates for the same validation predictions. Here exact match need not improve at every evaluation even as NLL falls. Teacher-forced NLL rewards probabilities assigned to the correct labels, while exact match requires every greedy decision and EOS to be correct on a generated sequence. Only the NLL enters the selection key.

Now freeze the winning choice and call `evaluate` once on its test split. Report NLL per evaluated token and counts per generated sequence. The two denominators answer different questions.

```
print("    LR  Update    Val NLL   Val exact   Val EOS")
for rate_9, update_9, scores_9 in sorted(
    history_9, key=lambda row: row[:2]
):
    print(
        f'{rate_9:6.3f} {update_9:7d} {scores_9["nll"]:10.6f} '
        f'{scores_9["exact_match"]:10.2%} '
        f'{scores_9["termination_rate"]:9.2%}'
    )

nll_9, selected_update_9, selected_rate_9 = best_key_9
print(f"\nSelected LR {selected_rate_9}, update {selected_update_9}")
print(f"Validation NLL {nll_9:.6f}")
test_metrics_9 = evaluate_9(
    winner_9, test_9, generation_max_new_tokens=cap_9
)
sequences_9 = test_metrics_9["sequences"]
exact_9 = round(test_metrics_9["exact_match"] * sequences_9)
ended_9 = round(test_metrics_9["termination_rate"] * sequences_9)
print(f'Test tokens {test_metrics_9["tokens"]}')
print(f'Test NLL {test_metrics_9["nll"]:.6f}')
print(f"Test exact match {exact_9}/{sequences_9}")
print(f"Test termination {ended_9}/{sequences_9}")
```

OUTPUT

```
    LR  Update    Val NLL   Val exact   Val EOS
 0.001       0   3.024102      0.00%    14.06%
 0.001      60   1.087083      1.56%    96.88%
 0.001     120   0.583314     31.25%   100.00%
 0.001     180   0.418732     37.50%   100.00%
 0.001     240   0.323457     51.56%   100.00%
 0.004       0   3.024102      0.00%    14.06%
 0.004      60   0.655319     20.31%    98.44%
 0.004     120   0.299161     57.81%   100.00%
 0.004     180   0.282064     64.06%   100.00%
 0.004     240   0.269765     59.38%   100.00%

Selected LR 0.004, update 240
Validation NLL 0.269765
Test tokens 352
Test NLL 0.260315
Test exact match 37/64
Test termination 64/64
```

In the saved run, learning rate 0.004 at update 240 gives the lowest validation NLL. Its validation exact-match rate at update 180 is higher than at 240. The specified rule still chooses 240 because the loss is lower there. The final test estimate applies to that single selected checkpoint under the same generation cap. Choosing a different model after seeing the test score would turn the test set into another selection set.

10. Detect information leakage

Keep the target shift and change which keys the decoder may read. With the properly shifted input for *red green blue*, the second input token is *blue*. The first label is also *blue*. If the first query can attend to the second input position, it can read the color it is supposed to predict. The next two color labels are exposed the same way. EOS is different. It appears in the labels only, so the final valid query has to predict it without reading a later EOS input.

Decoder position	1	2	3	4
Input	BOS	blue	green	red
Label	blue	green	red	EOS
Revealing input	position 2	position 3	position 4	none

Compare two fresh models. Both use the same default initialization from seed 2026, and both draw the same training minibatches from the same split. We use a separate copy of the companion repository. Run the original code in one Python process, edit only `target_allowed` in the copied Chapter 5 mask builder, and then start a second process in the copy. The second process imports both the model and the decoding helpers from that copy. This matters because the decoding module imports `build_masks` by name.

The score from a whole reference row and the score from a truncated reference prefix are two separate measurements. The first is the teacher-forced validation NLL requested by the exercise. Sum the `token_loss` numerators across batches, sum their non-PAD label counts, and divide once. For the second measurement, cut the reference input off just before the label would be revealed. At position i, keep input positions 1 through i. It still supplies the true past but removes the future. Compare both scores with greedy generation, which supplies only the model's previous predictions.

Both generation calls explicitly disable the cache. The cache assumes that adding a token leaves earlier states valid. Once the decoder can read future keys, adding a token may change those earlier states. Recomputing the prefix applies the experimental mask to the entire sequence available at that step.

The first part of the subprocess checks the source of both imports and reports the allowed keys for the first query. In a row with three real tokens followed by PAD, the original first query can read only its own key. The broken mask exposes all three real keys and still excludes PAD.

```
from shutil import copytree, ignore_patterns
import json
import subprocess
from tempfile import TemporaryDirectory
from textwrap import dedent

child_setup = dedent(r'''
    import json
    import random
    from pathlib import Path

    import torch

    from examples.ch05_transformer import model as architecture
    from examples.ch06_training import decoding
    from examples.ch06_training.data import (
        EOS, PAD, VOCAB, collate, make_splits, words,
    )
    from examples.ch06_training.training import (
        exact_matches, token_loss, train_step,
    )

    root = Path.cwd().resolve()
    for module in (architecture, decoding):
        assert Path(module.__file__).resolve().is_relative_to(root)
    assert decoding.build_masks is architecture.build_masks
    validity = torch.tensor([[True, True, True, False]])
    source_allowed, target_allowed = architecture.build_masks(
        validity, validity,
    )
    assert torch.equal(source_allowed[0, 0, 0], validity[0])
    first_query_keys = target_allowed[0, 0, 0].int().tolist()
''')
```

Use two helpers for validation. One computes the total loss when each query receives only its reference prefix. Within a batch, select only rows with a real label at that position so that each truncated input has no padding. The other helper collects the ordinary teacher-forced loss, generated exact matches, EOS terminations, and individual examples. An exact match includes EOS. A termination counts any generated EOS, even if it ends the row before all the colors have been generated.

```
child_prefix_score = dedent(r'''
    def reference_prefix_loss(model, batch, memory, source_allowed):
        loss_sum = 0.0
        for i in range(batch.target.shape[1]):
            active = batch.target[:, i] != PAD
            prefix = batch.decoder_input[active, :i + 1]
            logits = decoding.decode_prefix(
                model, prefix, memory[active], source_allowed[active],
            )[:, -1]
            total, _ = token_loss(logits, batch.target[active, i])
            loss_sum += total.item()
        return loss_sum
''')
```

```
child_validation = dedent(r'''
    @torch.no_grad()
    def check_validation(model, validation, cap, batch_size):
        model.eval()
        loss_sum = prefix_sum = 0.0
        count = exact = terminated = 0
        examples = []
        for start in range(0, len(validation), batch_size):
            batch = collate(validation[start:start + batch_size])
            logits = model(batch.source, batch.decoder_input)
            total, n = token_loss(logits, batch.target)
            loss_sum += total.item()
            count += n.item()

            memory, allowed = decoding.encode_source(model, batch.source)
            prefix_sum += reference_prefix_loss(
                model, batch, memory, allowed,
            )
            generated = decoding.greedy_decode(
                model, batch.source, max_new_tokens=cap, cached=False,
            )
            exact += exact_matches(generated, batch.target)
            terminated += generated.eq(EOS).any(dim=1).sum().item()
            for source, target, result in zip(
                batch.source, batch.target, generated,
            ):
                examples.append({
                    "source": words(source),
                    "reference": words(target, include_special=True),
                    "generated": words(result, include_special=True),
                })
        return {
            "full_nll": loss_sum / count, "prefix_nll": prefix_sum / count,
            "tokens": count, "exact": exact, "terminated": terminated,
            "examples": examples, "first_query_keys": first_query_keys,
        }
''')
```

Use 128 training sequences and 32 validation sequences, evenly split between lengths 3 and 4. The data seed is 2026. Hold out the test split. Train on a single CPU thread for 1,000 updates with minibatches of 32 and use the final update for both runs. Each model has width 64, four heads, FFN width 128, two layers per stack, zero dropout, and Adam with rate 0.001. Fix the generation cap at six for both models. The mask assignment is the only experimental change.

```
child_train = dedent(r'''
    torch.set_num_threads(1)
    torch.use_deterministic_algorithms(True)
    torch.manual_seed(2026)
    rng = random.Random(2026)
    train, validation, _ = make_splits(
        seed=2026, n_train=128, n_validation=32, n_test=32,
        min_length=3, max_length=4,
    )
    model = architecture.Transformer(
        len(VOCAB), len(VOCAB), d_model=64, n_heads=4, d_ff=128,
        n_encoder_layers=2, n_decoder_layers=2, dropout=0.0,
    )
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
    for _ in range(1000):
        batch = collate(rng.sample(train, 32))
        train_step(model, optimizer, batch)
    result = check_validation(model, validation, cap=6, batch_size=16)
    print(json.dumps(result))
''')
child_program = "\n".join((
    child_setup, child_prefix_score, child_validation, child_train,
))
```

Start both subprocesses with the same interpreter that runs the notebook. Python's `-E` flag ignores Python environment variables that could alter import resolution. The `-B` flag prevents writing bytecode files. The copy excludes bytecode caches and run artifacts so the child imports the source at its current revision in the temporary directory.

```
def run_mask_experiment(directory):
    completed = subprocess.run(
        [sys.executable, "-E", "-B", "-c", child_program],
        cwd=directory, capture_output=True, text=True,
    )
    if completed.returncode:
        raise RuntimeError(completed.stderr)
    return json.loads(completed.stdout)


with TemporaryDirectory(prefix="ch06-mask-study-") as temporary:
    copied_repo = Path(temporary) / "transformers-book"
    copytree(
        repo_root, copied_repo,
        ignore=ignore_patterns(
            ".git", ".venv", "__pycache__", "output", ".ipynb_checkpoints",
        ),
    )
    mask_source = copied_repo / "examples/ch05_transformer/model.py"
    original_source = mask_source.read_text(encoding="utf-8")
    original_line = (
        "target_allowed = causal[None, None] "
        "& target_valid[:, None, None, :]"
    )
    experimental_line = "target_allowed = target_valid[:, None, None, :]"
    assert original_source.count(original_line) == 1
    causal_run = run_mask_experiment(copied_repo)
    mask_source.write_text(
        original_source.replace(original_line, experimental_line, 1),
        encoding="utf-8",
    )
    broken_run = run_mask_experiment(copied_repo)
```

```
assert causal_run["first_query_keys"] == [1, 0, 0, 0]
assert broken_run["first_query_keys"] == [1, 1, 1, 0]
assert causal_run["tokens"] == broken_run["tokens"] == 144
assert abs(causal_run["prefix_nll"] - causal_run["full_nll"]) < 1e-6
print("Allowed keys for first query, including a PAD slot")
print("  Causal:    ", causal_run["first_query_keys"])
print("  Future keys", broken_run["first_query_keys"])
print("\nMask           Full NLL  Prefix NLL    Exact   EOS seen")
for name, row in (("Causal", causal_run), ("Future keys", broken_run)):
    n = len(row["examples"])
    print(
        f"{name:<13}  {row['full_nll']:>8.4f}  {row['prefix_nll']:>10.4f}"
        f"    {row['exact']:>2}/{n}     {row['terminated']:>2}/{n}"
    )

print("\nSame validation sources, decoded in both runs")
for base, leaky in zip(causal_run["examples"][:4], broken_run["examples"]):
    assert base["source"] == leaky["source"]
    assert base["reference"] == leaky["reference"]
    print(f"  Source:    {base['source']}")
    print(f"  Reference: {base['reference']}")
    print(f"  Causal:    {base['generated']}")
    print(f"  Future:    {leaky['generated']}\n")
```

```
OUTPUT
Allowed keys for first query, including a PAD slot
  Causal:     [1, 0, 0, 0]
  Future keys [1, 1, 1, 0]

Mask           Full NLL  Prefix NLL    Exact   EOS seen
Causal           0.8295      0.8295    12/32     32/32
Future keys      1.0391      6.6693     0/32     32/32

Same validation sources, decoded in both runs
  Source:    green green white green
  Reference: green white green green <eos>
  Causal:    green white green <eos>
  Future:    <eos>

  Source:    black white red red
  Reference: red red white black <eos>
  Causal:    red red white black <eos>
  Future:    <eos>

  Source:    blue red white red
  Reference: red white red blue <eos>
  Causal:    red white blue <eos>
  Future:    <eos>

  Source:    black yellow green red
  Reference: red green yellow black <eos>
  Causal:    red green yellow black <eos>
  Future:    <eos>
```

The causal model's two NLLs both round to 0.8295. Cutting off the future removes keys that the causal model was already forbidden to read. The broken model's NLL rises from 1.0391 to 6.6693 when we cut off the future, even though every preceding reference color is still correct. That jump measures how much the predictions relied on access to future inputs in this run.

Greedy decoding also feeds back its own choices. The causal model gets 12 of 32 full validation answers correct and the broken model gets none. Both runs produce EOS on every validation sequence, but the broken model emits it immediately on the four displayed rows. It stops without producing any of their colors. Termination tracks whether the model stops. An exact match also requires every color to be correct and in order.

11. Break and repair a cache (optional experiment)

Compare incremental decoding with the full-prefix decoder using the same model, encoded source, and input tokens. The small model uses float64, two decoder layers, two attention heads, and head width 4. Evaluation mode makes dropout inactive. The shorter source row has one padded column. Both decoding paths read the same encoder memory with the same source mask, so neither attends to that padded column.

Use the shifted reference input as a predetermined prefix. At the final column, the shorter row has `PAD`. For this comparison feed a repeated `EOS` there in both paths because `CachedDecoder.step` rejects `PAD` inputs. The test compares logits on supplied prefixes, so no token selection or stopping rule can change the tokens being compared.

```
model_11 = make_ch06_checks_model()
batch_11 = collate([(3, 4, 5, 6), (6, 4, 3)])
prefix_11 = batch_11.decoder_input.masked_fill(
    batch_11.decoder_input == PAD, EOS
)
memory_11, source_allowed_11 = decoding.encode_source(
    model_11, batch_11.source
)
print("Sources            ", batch_11.source.tolist())
print("Allowed source keys", source_allowed_11[:, 0, 0].int().tolist())
print("Supplied prefixes  ", prefix_11.tolist())
```

```
OUTPUT
Sources             [[3, 4, 5, 6], [6, 4, 3, 0]]
Allowed source keys [[1, 1, 1, 1], [1, 1, 1, 0]]
Supplied prefixes   [[1, 6, 5, 4, 3], [1, 3, 4, 6, 2]]
```

At each length t, pass just the new input column to the cache. The reference call sees the same first t columns together, so its last logit row is the prediction after consuming that same new token. Compare the full logit vectors for both examples. Each layer's self cache should have key and value shapes $[2, 2, t, 4]$ as its token axis grows.

For the broken variant, subclass `CachedDecoder` and reset its position counter before each call. The layer caches still grow, but every new input receives the positional vector for index 0. Run the repaired comparison with a fresh canonical decoder. A fresh cache recomputes the earlier keys and values with the correct positions.

```
class ZeroPositionDecoder(decoding.CachedDecoder):
    def step(self, token):
        self.position = 0
        return super().step(token)

def compare_prefixes_11(cache_class):
    cache = cache_class(model_11, memory_11, source_allowed_11)
    rows = []
    for length in range(1, prefix_11.shape[1] + 1):
        token = prefix_11[:, length - 1:length]
        incremental = cache.step(token)
        full = decoding.decode_prefix(
            model_11, prefix_11[:, :length], memory_11, source_allowed_11
        )[:, -1]
        for key, value in cache.self_kv:
            assert key.shape == value.shape == (2, 2, length, 4)
        max_error = (incremental - full).abs().max().item()
        close = torch.allclose(incremental, full, atol=1e-12, rtol=1e-12)
        rows.append((max_error, close))
    return rows
```

```
clean_11 = compare_prefixes_11(decoding.CachedDecoder)
broken_11 = compare_prefixes_11(ZeroPositionDecoder)
repaired_11 = compare_prefixes_11(decoding.CachedDecoder)

assert all(close for _, close in clean_11 + repaired_11)
first_failure_11 = next(
    length for length, (_, close) in enumerate(broken_11, 1)
    if not close
)
assert first_failure_11 == 2
print("Prefix length   Baseline error   Zero position   Repaired error")
for length, rows in enumerate(zip(clean_11, broken_11, repaired_11), 1):
    baseline, broken, repaired = (row[0] for row in rows)
    print(f"{length:13d}   {baseline:14.3e}   {broken:13.3e}   "
          f"{repaired:14.3e}")
print(f"First failing prefix length  {first_failure_11}")
```

```
OUTPUT
Prefix length   Baseline error   Zero position   Repaired error
            1        0.000e+00       0.000e+00        0.000e+00
            2        0.000e+00       1.632e-01        0.000e+00
            3        0.000e+00       3.174e-01        0.000e+00
            4        2.220e-16       1.934e-01        2.220e-16
            5        2.359e-16       9.305e-02        2.359e-16
First failing prefix length  2
```

The first input belongs at position index 0 in both paths. The bug becomes visible at length 2, when the new token belongs at index 1 but receives index 0 again. The wrong positional vector changes its representation before projection to queries, keys, and values. The output logits then differ, and the appended incorrect keys and values persist into later steps. A first-token check cannot exercise the position advance.

12. Time equal work (optional experiment)

Use one fixed CPU source and a five-token, nonpadding target prefix. Both decode paths return logits for the position that consumes the final supplied input. The full-prefix path computes the decoder over all five supplied tokens and we need its final row. The cached path has already consumed the first four and adds just the final input. Before timing, check that those two logit rows agree.

`benchmark` reports four separate median times in milliseconds. Source encoding creates encoder memory and its padding mask. Cached target prefill starts with that memory, prepares the static cross-attention keys and values, and steps through the first four supplied inputs to build the self-attention cache. The later full-prefix timing uses the saved encoder memory and recomputes decoder states across the prefix. The later cached timing starts with a newly prefilled cache each repetition, so the measured step always consumes the fifth input at index 4. The cache construction and prefill happen outside that measured step.

```
model_12 = make_ch06_checks_model()
batch_12 = collate([(3, 4, 5, 6)])
source_12, prefix_12 = batch_12.source, batch_12.decoder_input
assert prefix_12.shape == (1, 5) and not prefix_12.eq(PAD).any()

memory_12, allowed_12 = decoding.encode_source(model_12, source_12)
cache_12 = decoding.CachedDecoder(model_12, memory_12, allowed_12)
for column_12 in prefix_12.split(1, dim=1):
    last_incremental_12 = cache_12.step(column_12)
full_12 = decoding.decode_prefix(
    model_12, prefix_12, memory_12, allowed_12
)
torch.testing.assert_close(
    last_incremental_12, full_12[:, -1], atol=1e-12, rtol=1e-12
)
print("Source IDs:", source_12.tolist())
print("Fixed target input IDs:", prefix_12.tolist())
print("Final-input logits agree to float64 tolerance.")
```

OUTPUT

```
Source IDs: [[3, 4, 5, 6]]
Fixed target input IDs: [[1, 6, 5, 4, 3]]
Final-input logits agree to float64 tolerance.
```

```
previous_threads_12 = torch.get_num_threads()
try:
    torch.set_num_threads(1)
    timings_12 = [
        decoding.benchmark(model_12, source_12, prefix_12, repeats=25)
        for _ in range(2)
    ]
finally:
    torch.set_num_threads(previous_threads_12)

timing_columns_12 = (
    ("Source encoding", "source_encode_ms"),
    ("Cached target prefill", "cached_target_prefill_ms"),
    ("Full-prefix decoder", "full_prefix_next_token_ms"),
    ("Cached final input", "cached_next_token_ms"),
)
print(f"CPU   {next(model_12.parameters()).dtype}   "
      f"{timings_12[0]['threads']} thread   25 repeats per run")
print("Median milliseconds per phase        Run 1     Run 2")
for label, key in timing_columns_12:
    left, right = (result[key] for result in timings_12)
    assert all(math.isfinite(ms) and ms > 0 for ms in (left, right))
    print(f"{label:32s}  {left:7.3f}   {right:7.3f}")
```

OUTPUT

```
CPU   torch.float64   1 thread   25 repeats per run
Median milliseconds per phase        Run 1     Run 2
Source encoding                     0.194     0.191
Cached target prefill               0.653     0.643
Full-prefix decoder                 0.323     0.326
Cached final input                  0.163     0.161
```

Fixing the prefix gives the two paths the same inputs and makes the comparison about computing the same output. Early `EOS` in a generation benchmark could end one path after fewer steps. Different continuations could change prefix lengths, output counts, and the amount of attention work. Charging source encoding to just one path would compare different timing boundaries. Compare the two later-token medians separately from setup. On this tiny model, Python overhead and the copies from appending to the cache can be a noticeable part of the time. The medians apply to this CPU, model, and fixed prefix length.